

Report

Sahaj Buttar

CSC-228

12/10/2025

Report

The main goal for me on this project was to show how the same dataset can be shown using many different data structures. The data structures that could do this: Binary Search Trees, Doubly Linked List, Stack and Queue, and the Graph. The project just mainly shows how the data can move through different data structures.

The Binary module mainly stores integer values in a way similar to a tree structure. It mainly shows how the left child of a node basically has less values than the node while the right side has a greater value. This mainly shows the insertion, deletion, and the main search operations. The recursive functions were the ones mainly implemented for insertion and deletion. The deletion mainly helps with removing the node with one child or removing the node with two children. In the removing case, the node's value is usually replaced by its in-order successor and then it is deleted. The traversals were mainly implemented in three orders. Starting with the first one, the in-order. In-order traversal goes to the left subtree, then it goes to the current node and then to the right subtree. The right subtree does the values in a sorted order. Pre-order traversals go to the current nodes first, then to the left and then the right. This helps with it copying the tree and also understanding the structure of it and the process of each in general. In conclusion, the module mainly helps with the benefits of data structures. At first doing the trees with data was tough as it was giving some problems with the code but it eventually was able to help show how the traversal functions were able to show the right values and that the structure was working properly.

The Double Linked List was helpful because it shows the Binary Search Tree in more of a way in a sequence. The node usually contains like pointers to help allow the traversal to go in both directions. This is helpful if BST data is converted into a linear structure. The BST values are exported to the Double Linked List and this is using the in-order traversal which helps with the order being sorted. The Double Linked List does allow insertion and deletion at both of the ends which makes it efficient. This is good and useful because single-linked lists cannot do this. With the Double Linked List, I also implemented stack and queue abstractions. The stack and queue are used to show the main concept of data abstractions. I tested the Double Linked List with the rest to make sure it was working and it was able to show the sorted list correctly and also made sure that the BST value was correctly exported and this helped me mainly make sure that it was working. The stack and queue operations also worked efficiently and this was able to help make sure everything was in place and sorted.

The Graph module basically sees the Doubly Linked Lists node as a vertex in an undirected graph. I added the edges to mainly show the connection between the values and this helps show the simplicity of the connection. The adjacency list is mainly shown for using the memory. The BFS is mainly shown for the connection. I tested the graph and the testing was able to show the graph's structure and connection. The graph shows the main difference between these structures and that is helpful for many reasons which includes showing the main relationships between nodes.

In conclusion, the project shows the trade offs between usability and complexity of the time. The BST was useful for searching quickly and is good for traversals that are ordered. For sequencing, the Double Linked Lists do a good job and are good for insertion and deletion. Stack

and Queue mainly were helpful for data abstraction and the graph was helpful for memory. In all, this project was tough as implementing all these data structures was not easy and complex.